

Codiste-AIDLC — Optimization & Adoption Plan

v0.1

- **Author:** Varshil Gandhi (varshil.g@codiste.in)
 - **Date:** 2026-05-07
 - **Context:** Builds on the SDLC-vs-AIDLC e-commerce experiment and the brownfield recommendation doc. This is the *internal adoption + productization plan* for AI-DLC at Codiste — what we keep from AWS, what we change for our team profile, and how we get from a single experiment to a company-wide standard.
-

TL;DR

- AWS AI-DLC ([awslabs/aidlc-workflows](#)) is open, methodology-only, and ships a formal extension system. Codiste's job is **not to fork it — extend it**. Our edge is encoding *Codiste's playbook* (regulated verticals, agent specialization, BDE → Dev handoff) on top of the bones AWS already shipped.
 - Five concrete optimizations:
 - **(1)** Pre-Inception Profile Wizard (project type + role → pipeline variant)
 - **(2)** GitHub-as-the-spine (UoW = branch, gate = PR) **(3)** Role-based tracks for non-tech (BDE, Product) alongside tech
 - **(4)** Mandatory `grill-me` Inception with a new Gate #0
 - **(5)** Standardized tooling baseline (Claude Desktop layout, worktrees, output modes).
 - Lessons from the e-commerce greenfield experiment (the 9 post-Gate-#5 fix commits) are codified into hard rules, not soft guidance — they become enforced in tooling.
 - **Productization path:** internal v0.1 (4 weeks) → 3-project battle test (8 weeks) → external preview in BDE pitches (Q3) → marketed framework with case studies (Q4).
 - Single big idea: **The audit trail is the deliverable**. For Codiste's regulated-vertical clients (FinTech, RegTech, HIPAA), the AI-DLC sign-off ledger is not overhead — it is billable evidence of process, and the differentiator vs. agencies that just "use AI."
-

1. The Codiste opportunity (why this fits us specifically)

Codiste positions as an **"AI Agent Development Studio"** working with funded startups in regulated verticals — FinTech, RegTech, PropTech, MarTech, AdTech — with explicit compliance frameworks (SOC 2, ISO 27001, GDPR, HIPAA, EU AI Act, FINRA, PCI DSS, CCPA). Three things make AI-DLC fit Codiste better than most agencies:

1. **Compliance-heavy clients reward audit trails.** AI-DLC's sign-off ledger turns into compliance evidence by default. Clients pay extra for that with one auditor.

2. **Funded-startup clients pay for speed *and* accountability.** AI-DLC's gates are billable evidence that "we shipped fast and we shipped responsibly" — which is exactly the Codiste claim of 62% faster TTM with reputation-grade delivery.
3. **Senior-only team lets us run the pod model at scale.** AI-DLC's Tech Lead + Dev pod assumes both seats are competent. Codiste already operates that way; most agencies don't.

The risk: AWS's framework is generic. **Codiste's edge is in vertical specialization and role coverage** — our customization should encode our actual playbook, not just rename theirs. If we extend it correctly, "Codiste-AIDLC" becomes a marketed asset, not a process tax.

2. What we keep from AWS AI-DLC (don't reinvent)

The upstream `awslabs/aidlc-workflows` already ships:

- **3 phases:** Inception (WHAT/WHY) → Construction (HOW) → Operations (deploy/monitor)
- **Adaptive workflow principle:** "the workflow adapts to the work, not the other way around"
- **Tier system:** greenfield / feature / bugfix — already wired into our local `CLAUDE.md`
- **Pod sign-off:** 1 Tech Lead + 1 Dev, 5 gates — already configured in our `pod.md` (Chintan Bhai + Varshil)
- **Multiple sign-off mechanisms:** in-file markdown signature, **PR comment with `aidlc-signoff: <gate-id>`**, commit-message trailer, external-tool URL
- **Formal extension system:** `aws-aidlc-rule-details/extensions/` with one rules file + one opt-in prompt file per extension; teams can add new categories
- **Multi-IDE support:** Kiro, Amazon Q Developer, Cursor, Cline, Claude Code, GitHub Copilot, OpenAI Codex, others

Implication: Codiste-AIDLC is a *set of extensions and conventions on top of AWS's framework*, not a parallel product. We push our customizations as new directories under `extensions/codiste-*/`, not as a hard fork. Easier to maintain, easier to upstream parts later, easier to defend against "you just renamed AWS's thing."

3. The 5 Codiste optimizations

3.1 Pre-Inception "Profile Wizard" (combines Idea 1 + Idea 3)

Problem. AWS's adaptive workflow exists but is implicit — the model decides what to skip. Inconsistent across sessions. The greenfield MVP showed this: every developer needs to make the same routing decisions in their head.

Proposal. Add **Stage 0 — Profile Wizard** that runs *before* Inception. Two structured questions via the `AskUserQuestion` tool:

1. **Project type:** Greenfield MVP / Brownfield extension / Brownfield modernization / Pitch-only (BDE) / Spike

2. **Primary requester role:** Founder / Product Manager / BDE / Tech Lead / Frontend Dev / Backend Dev / QA / DevOps

The combination routes to a **pipeline variant**:

Project type × Role	Pipeline variant
Greenfield MVP × Tech Lead	Full AI-DLC (what we just ran)
Brownfield × Backend / Frontend Dev	AI-DLC Brownfield Lite (per separate doc)
Pitch × BDE	AI-DLC Pitch Track (new — see §3.3)
Idea-stage × Product Manager	AI-DLC Product Track (new — see §3.3)
Spike × any Dev	Vibe coding, no AI-DLC (skip the whole thing)
Bugfix × any Dev	Existing tier=bugfix path (already in repo)

Output: a **profile token** stamped into every artifact (e.g. `profile: greenfield/MVP/tech-lead`) that downstream stages and gates use for their own gating.

Why this matters. AWS's "adaptive workflow" exists but isn't explicit. Codiste makes it a 30-second decision at the start, not a model judgment call mid-flow. Decisions made up front are auditable; decisions made silently by a model are not.

3.2 GitHub-as-the-spine for UoWs and gates (Idea 2 — extended)

Current state. Our local AI-DLC supports `aidlc-signoff: <gate-id>` PR comments and commit trailers as sign-off mechanisms. We don't enforce them — sign-offs are still done as in-file markdown edits.

Proposal — three rules to make GitHub the source of truth:

1. **UoW = feature branch.** Every UoW spin-up automatically runs `git checkout -b uow/NN-<slug>`. All Construction commits land there. Codiste convention enforced via a slash command or hook.
2. **Gate = PR.** A signed sign-off is a PR comment by the named pod members containing `aidlc-signoff: gate-N`. PR merge = gate sealed. No more manual markdown signatures.
3. **MCP-enforced.** GitHub MCP installed at session start (we did this on the SDLC project). Tooling enforces:
 - Construction stage cannot run if no feature branch exists
 - Gate cannot be marked signed if PR is not approved + merged
 - Sign-off requires both pod members' GitHub accounts to comment-approve

Bonus: GitHub becomes the audit log for compliance. A SOC 2 auditor asks "show me the change-approval evidence." We answer: "here's the PR with two-person sign-off, immutable, tied to commits,

tied to AI-DLC artifacts in the same PR, with the AI-DLC profile token in the description." That maps cleanly onto FinTech / RegTech / HIPAA reporting requirements.

This is **the** Codiste piece — the most important customization for our vertical positioning. Other AI-DLC users can skip it; for us it's the marketable difference.

3.3 Role-based pipeline variants (Idea 3 — extended to non-tech)

The existing tier system (greenfield/feature/bugfix) only addresses **scope**. Add **role tracks** that address **audience**:

Track	Audience	What runs	What's produced
Pitch Track	BDE, Sales	Idea brief → discovery questions → market validation → competitive scan → pitch deck outline → effort/cost estimate	Client pitch deck + 1-page proposal + Discover-phase scope
Product Track	Product Manager	Idea → user research synthesis → PRD → roadmap → success metrics	PRD, roadmap, success metrics — handed off to Tech Track
Tech Track (default)	Tech Lead, Full-stack Dev	Full Inception → Construction → Operations (current AI-DLC)	Working software
Frontend-Dev Track	Frontend specialist	Subset: design intake → component spec → implementation → visual regression	UI features, no backend agent stages
Backend-Dev Track	Backend specialist	Subset: data model → API design → implementation → contract tests	API surfaces, no UI stages
QA Track	QA engineer	Test plan → test cases → automation → regression suite	Test artifacts, traceable to UoW IDs
DevOps Track	Platform engineer	IaC scaffold → deployment plan → observability → runbook	Deployment + ops bundle

All tracks share the **same gate model and audit format**. A BDE's pitch artifact and a backend dev's API design carry the same AI-DLC sign-off signature. **That's the productization unlock — one common ledger across the company.**

The Pitch Track is the most strategically important addition. If a BDE produces a client pitch deck *with the AI-DLC sign-off chain attached*, the deck stops being a sales artifact and becomes a partial Discover-phase deliverable. This is exactly Codiste's "Discover → Prototype → Launch → Iterate" methodology made operational.

3.4 Skill-augmented Inception with grill-me (Idea 4)

Problem. Inception's biggest failure mode is "looks complete but the user actually didn't share the constraint that matters." The greenfield MVP showed this directly — 9 fix commits after Gate #5

because real-world constraints surfaced too late (multi-format tool-call parsing, CSP issues, session isolation, etc.). Every gate signed; integration reality unsigned.

Proposal. At the start of Inception, mandatorily invoke `grill-me` (or an equivalent structured-questioning skill) seeded with the project profile from §3.1. The skill produces a **shared understanding doc** — explicit assumptions, known unknowns, "what could break this" — *before* any other Inception artifact is written.

Add a new gate:

Gate #0 — Shared Understanding. Tech Lead signs off on the shared-understanding doc before Business Requirements stage starts.

This is the cheapest possible insurance against late-stage churn. One additional gate, ~30 minutes of structured Q&A, saves multi-day rework downstream. It also forces the Tech Lead to engage early rather than only at gate-review time — which is the most common reason gates get rubber-stamped.

3.5 Standardized tooling baseline (Claude Desktop tips, operationalized)

The 11 tips collected from the Avthar Claude Desktop video aren't a tip list — they're a **default working environment**. Codiste-AIDLC ships them as the standard layout:

Convention	Codiste rule
Three-column layout (chat / preview / terminal+plan)	Default for any active build session
Worktree-per-session (checkbox)	Mandatory for parallel UoWs — one Claude per UoW, no shared working tree
Plans pane with inline comments	Plan revisions must be commented inline, not re-prompted from scratch
AskUserQuestion in Inception	Mandatory in Profile Wizard (§3.1) and during <code>grill-me</code> (§3.4)
Side chats (Cmd+;)	Use for "would-this-work-later" tangents; main session stays clean
Diff viewer + line-level comments	Code Review (Gate #4) uses this exclusively — no separate markdown review docs
Sessions grouped by repo	Default sidebar setting for all engineers
Cloud sessions for long jobs	Long Inception research / overnight builds run in cloud sessions
Output modes per task	Verbose for debugging, Summary on session resume after a break
Skills/plugins parity (desktop ↔ CLI)	Codiste skill bundle (<code>grill-me</code> , <code>aidlc-stage-*</code> , <code>signoff</code>) installed once, runs everywhere
Multi-session (up to 4 quadrants) + ask Claude to merge	Reach for this when 2–4 truly independent UoWs; don't force-fit dependent work

These belong in the company-level `CLAUDE.md` template alongside AI-DLC instructions, not in a separate "tips doc." Tooling baseline is part of the methodology.

4. The 8 lessons from the prior project — codified into hard rules

Direct port of your 8 lessons into Codiste-AIDLC enforcement (so they don't have to be re-learned per project):

Lesson from prior project	Codification	Where it lives
1. Clear MVP understanding before starting	Profile Wizard mandatory (Stage 0)	§3.1
2. Focus on Inception	Mandatory grill-me + Tech Lead signs new Gate #0	§3.4
3. Define conditions early (acceptance criteria)	Business Requirements gate must include explicit acceptance criteria, not only stories	Inception template
4. Careful sign-off (don't rubber-stamp)	Gate cannot pass without integration-shape tests + GitHub PR approval	§3.2 + §3.4
5. Use Opus throughout	Default model in shared <code>CLAUDE.md</code> = <code>claude-opus-4-7</code> ; Sonnet only on >100k LOC tasks	Tooling baseline
6. Run smoke tests	New mandatory Smoke-Test stage between Construction Gate #4 and Operations Gate #5	§5 reference flow
7. Temperature = 0 for determinism	Standard config in shared <code>CLAUDE.md</code>	Tooling baseline
8. Test with real systems before sign-off	Gate #5 requires real-shape integration test, not just unit tests passing	§3.2

The headline finding from the SDLC-vs-AIDLC report — 9 post-Gate-#5 fix commits — was caused by gaps in lessons 4, 6, 7, 8. Codifying them removes the human-discipline tax.

5. Codiste-AIDLC reference flow (full Tech Track)

```

[Stage 0 – Profile Wizard]    project type + role → variant routed
    ↓
[Gate #0 – Shared Understanding]    grill-me output, Tech Lead signs (PR-merged)
    ↓
[INCEPTION]    PRD-delta · Domain Story · Use Cases · Capability Map · NFR
    - Gate #1 (PR-merged, both pod members)
    ↓
[ARCHITECTURE]    Logical · Domain Model · Stack (skip if brownfield)
    - Gate #2 (PR-merged)
    ↓
[CONSTRUCTION]    Per-UoW: feature branch → code → review → diff comments → tests
    - Gate #3 (functional design) + Gate #4 (code review) per UoW (PR-merged)
    ↓
[SMOKE-TEST]    real-shape integration tests on staging    ← NEW
    ↓
[OPERATIONS]    Deploy · Observe · Runbook
    - Gate #5 (PR-merged + production smoke pass)
    ↓
[CLOSEOUT]    Audit bundle exported as compliance artifact

```

Track variants collapse stages: Pitch Track only runs Stages 0 + an abridged Inception. Frontend-Dev Track skips backend Architecture stages. QA Track runs the test artifacts in parallel with Construction. The reference flow above is the *Tech Track default*.

6. Productization roadmap (internal → external)

Phase	Target timeline	Goal	Exit criterion
6.1 Internal v0.1 (now)	4 weeks	Codiste-AIDLC repo as extension of <code>awslabs/aidlc-workflows</code> ; Profile Wizard; GitHub MCP enforcement; all tracks scaffolded	One real client project run end-to-end on it
6.2 Internal v0.5 (battle test)	8 weeks	3 client projects on Codiste-AIDLC; lessons feed back; BDE/Product tracks tested	<2 post-Gate-#5 fix commits per project (vs. our 9 on the e-commerce MVP)
6.3 External preview	Q3	Codiste-AIDLC referenced in BDE pitches; clients see audit trail in Discover phase	First client signs <i>because</i> of the methodology, not despite it
6.4 External productization	Q4	Open-source the non-vertical parts (Codiste fork of <code>aidlc-workflows</code>); proprietary vertical extensions stay internal	First external contributor PR merged; first case study published

7. Risks & open questions

1. **Tooling parity drift.** GitHub MCP, Codiste skill bundle, and shared `CLAUDE.md` live in 2+ places per engineer. → Single `codiste-aidlc-bootstrap` skill that installs everything (MCP, skills, `CLAUDE.md` template, output mode defaults).
2. **Non-tech adoption.** BDE / Product team may resist a structured pipeline. → Pitch Track must produce *measurably better* pitch decks than the current freeform method. Don't ship until that's true. Pilot with one BDE on one real client opportunity.
3. **Sign-off bottleneck on Tech Lead.** Currently Chintan Bhai is the only signer; substitute matrix in `pod.md` is empty (`(none)`). → At company scale, define tier-based delegation (e.g., bugfix tier can be signed by any senior dev; greenfield requires Tech Lead).
4. **Vertical specialization vs. generic methodology.** AWS's framework is generic; our edge is FinTech/RegTech/HIPAA. → Each vertical needs its own extension under `extensions/codiste-fintech/` , `extensions/codiste-hipaa/` . Build one as a proof point in v0.5.
5. **Audit storage & aggregation.** Where do compliance artifacts live across many client projects? → Each project's GitHub repo + a Codiste-internal audit aggregator. Open question: do we build that aggregator now, or rely on GitHub's API + manual export?
6. **Model-cost economics.** Defaulting to Opus everywhere (Lesson 5) is the right correctness choice but raises per-project model cost. → Track cost-per-gate as a metric; revisit Sonnet-for-Construction policy after 3 projects of evidence.
7. **Rubber-stamp risk persists.** Even with Gate #0 and integration tests, gates can still be signed unthinkingly. → Spot-audit policy: Tech Lead's manager (or peer Tech Lead) randomly reviews 1 in 10 sealed gates, with consequences for rubber-stamping. Cultural, not technical.

8. One-line summary

AWS gave us the bones; Codiste's job is to put the playbook on top. Profile Wizard at the front, GitHub-spined sign-offs in the middle, role-based tracks across the team, and a tooling baseline that ships standard with every project — that's the v0.1 of Codiste-AIDLC. Internal in 4 weeks, battle-tested in 12, marketed by Q4.

Appendix A — Source references

- **AWS AI-DLC repo:** <https://github.com/awslabs/aidlc-workflows>
- **AWS AI-DLC announcement blog:** <https://aws.amazon.com/blogs/devops/ai-driven-development-life-cycle/>

- **AWS AI-DLC re:Invent 2025 talk (DVT214):** <https://aws.amazon.com/blogs/devops/open-sourcing-adaptive-workflows-for-ai-driven-development-life-cycle-ai-dlc/>
- **Sample AI-DLC platform (multi-agent reference impl on Bedrock):**
<https://github.com/aws-samples/sample-ai-driven-development-lifecycle-platform>
- **Codiste website:** <https://www.codiste.com/>
- **Prior internal docs:** `SDLC-vs-AIDLC-report.md` , `Brownfield-AIDLC-vs-SDLC-recommendation.md` (this folder)
- **Local AI-DLC instance referenced:** `/home/user/Documents/Project/ECommer-AIDLC/`
(`CLAUDE.md`, `pod.md`, `aidlc-docs/`)